

A - Priority Eviction

An integral part of any computer system is memory. It is used to store and retrieve information, intermediate results, and metadata about other data. Naturally, optimizing how memory is used becomes a crucial task in designing large-scale systems.

A computer can be abstracted as a machine consisting of a CPU, memory, and peripherals. The CPU has its own small internal memory (registers, and typically a hardware cache) that it uses to perform computations. This internal memory is extremely fast, but also extremely limited in size — often just a few megabytes, compared to the gigabytes of data a program may need to work with. This is why a separate, much larger main memory (RAM) exists: to hold everything the internal memory cannot. The trade-off is that main memory, being larger and physically farther from the CPU, is significantly slower to access.

This creates a tension: we want the speed of small memory but the capacity of large memory. The resolution relies on an observation about how programs behave in practice, known as *locality of reference* — programs tend to access the same small set of data repeatedly over short windows of time (think of a web browser repeatedly redrawing the same handful of open tabs, or an OS repeatedly reading the same few files). This means that if we keep *recently or frequently used* data in a small, fast intermediate store — a *cache* — sitting between the CPU and main memory, most accesses can be served quickly without going all the way to main memory.

Since a cache is limited in size, it will eventually fill up. At that point, before new data can be brought in, the system must decide which existing data to discard — a decision governed by an *eviction policy*. Formalizing and implementing such a policy is the objective of this problem.

Data is typically moved between main memory and a cache in fixed-size, uniquely identifiable blocks called *pages*. In this problem, a page is simply an abstract, indivisible unit of data, identified by an integer ID, that can either be resident in the cache or not.

Problem Statement

There are M distinct pages, identified by IDs 0 to $M - 1$, that can be cached. The cache can hold at most N pages at a time, where $N < M$. Each page, when accessed, is assigned a priority value. When the cache is full and a new page needs to be inserted, the page with the **lowest** priority currently in the cache must be evicted to make room.

Design and implement a system that supports the following operations efficiently:

- **accessPage(pageID)**: Access the given page. If it is already in the cache, this counts as a hit; if it is not in the cache, it must be inserted, evicting a page first if the cache is full.
- **evict()**: Remove the page with the lowest priority from the cache.
- **removePage(pageID)**: Explicitly remove a page from the cache, if present.

Your implementation must use exactly the following three data structures, each fulfilling a distinct role:

- A **doubly linked list**, to hold the pages currently resident in the cache.

- A **min-heap**, to efficiently determine the lowest-priority page in the cache.
- An **array** of size M , indexed by page ID, to provide direct access to a page's location in the cache without the use of a hashmap.

Sample Test Case

Let $M = 6$ (page IDs 0 through 5) and cache capacity $N = 3$. Assume each page starts at priority 1 when first accessed, and each subsequent **accessPage** hit increases its priority by 1; ties in priority are broken by evicting the page with the *smaller* ID.

Step	Operation	Result	Cache (ID:priority) after step
1	accessPage(0)	MISS	{0:1}
2	accessPage(1)	MISS	{0:1, 1:1}
3	accessPage(2)	MISS	{0:1, 1:1, 2:1}
4	accessPage(0)	HIT	{0:2, 1:1, 2:1}
5	accessPage(3)	MISS, evicts page 1	{0:2, 2:1, 3:1}
6	accessPage(1)	MISS, evicts page 2	{0:2, 3:1, 1:1}
7	removePage(2)	False (already evicted)	{0:2, 3:1, 1:1}
8	evict()	removes lowest priority (tie 1 vs. 3 $\rightarrow$ ID 1)	{0:2, 3:1}
9	removePage(3)	True	{0:2}



Figure 1: Picture unrelated btw (TA's fav anime tho)

B - Young Tableau

A **Young tableau** is an $n \times n$ matrix Y such that the entries in each row are sorted from left to right, and the entries in each column are sorted from top to bottom:

$$Y[i, j] \leq Y[i + 1, j] \quad \text{and} \quad Y[i, j] \leq Y[i, j + 1]$$

Unoccupied cells in the tableau are filled with ∞ . An $n \times n$ Young tableau can store up to $N = n^2$ finite real numbers and can be viewed as an alternative implementation of a min-priority queue.

- (a) **Matrix Structure:** Draw an initially empty 4×4 Young tableau and show its configuration after sequentially inserting the following elements:

9, 16, 3, 2, 4, 8, 5, 14, 12

- (b) **Extract-Min Operation:** Provide pseudocode for an algorithm $\text{EXTRACT-MIN}(Y)$ that extracts and returns the minimum element from an $n \times n$ Young tableau. Show that it restores the tableau property in $O(\sqrt{N})$ time, where $N = n^2$.
- (c) **Insertion Operation:** Provide pseudocode for an algorithm $\text{INSERT}(Y, x)$ that inserts key x into an $n \times n$ Young tableau. Show that it maintains tableau invariants in $O(\sqrt{N})$ time.
- (d) **Search Operation:** Design an algorithm $\text{SEARCH}(Y, x)$ that checks if $x \in Y$ in $O(\sqrt{N})$ worst-case time.
- (e) **Tableau Sort vs. Heap Sort Analysis:** Standard Heap Sort processes N elements with a priority queue operation cost of $O(\log N)$, yielding a total running time of $O(N \log N)$.
- Show that sorting $N = n^2$ elements using a Young tableau by performing N INSERT operations followed by N EXTRACT-MIN operations requires $O(N^{3/2})$ time by replacing $O(\log N)$ with $O(\sqrt{N})$.
 - Express $O(N^{3/2})$ in terms of the matrix dimension n to show that the sorting runtime is $O(n^3)$.
- (f) **Implement the algorithm in Python/C++/Java:** Write appropriate edge-cases and test the correctness of your code.